

Metody numeryczne

Wersja robocza
20 lutego 2026

Konrad J. Kossacki

W niniejszym skrypcie omówione są przykładowe metody numerycznego rozwiązywania wybranych problemów matematycznych spotykanych w fizyce. Materiał został przygotowany dla uczestników ćwiczeń do wykładu **Metody numeryczne**, a więc osób posiadających już umiejętność samodzielnego pisania programów.

Początkujący programiści powinni wybrać na przykład przedmiot **Programowanie i metody numeryczne**.

Podczas ćwiczeń studenci mogą używać dowolnego języka programowania, w którym potrafią sprawnie programować i będą w stanie samodzielnie znajdować i usuwać formalne błędy w swoich programach.

Rozdział 1

Dokładność numeryczna

Ćwiczenie

Epsilon maszynowy to najmniejsza liczba, która dodana do jedynki da wynik różny od jeden. Znajdź taką liczbę na swoim komputerze.

Ćwiczenie

Proszę napisać program który doda do siebie n razy liczbę δx , przy czym liczba n ma wartości: 10, 100, 1000, 10000, 100000, 1000000). Program ma narysować wykres odchylenia błędu wyniku sumowania od prawidłowego rezultatu w zależności od n . Jakie są odchylenia dla $\delta x = 0.1$ oraz 0.125.

Ćwiczenie

Proszę obliczyć kolejne wartości ciągu

$$x_{n+1} = 13/3x_n - 4/3x_{n-1}, \quad (1.1)$$

gdzie $x_0 = 1, x_1 = 1/3$. Jak mają się one do prawdziwych wartości $x_n = 1/3^n$?

Ćwiczenie

Proszę zbadać przy użyciu wykresu szybkość zbieżności do granicy szeregów:

$$a_n = (n+1)/n, \lim a_n = 1 \quad (1.2)$$

$$b_n = (1 + 1/n)^n, \lim b_n = e \quad (1.3)$$

$$c_{n+1} = 1/2c_n + 1/c_n, \lim c_n = \sqrt{2}, c_0 = 2 \quad (1.4)$$

$$d_{n+1} = \frac{d_{n-1}d_n + 1}{d_{n-1} + d_n}, \lim c_n = 1, d_0 = 0, d_1 = 2. \quad (1.5)$$

Proszę zrobić wykres, najlepiej logarytmiczno logarytmiczny dla $0 \leq n \leq 100$.

Prosty program rysujący wykres na ekranie, napisany w języku Python może mieć postać:

```
import matplotlib.pyplot as plo
import math

def funkcja(x):
    pi=math.pi
    a1=x/180.0*pi
    a4=math.sin(a1)
    return a4

x1=0.0
xp=180.0
n=200
xx=[]
yy=[]

for i in range(0,n+1):
    x=i*(xp-x1)/n+x1
    xx.append(x)
    y=funkcja(x)
    yy.append(y)

plo.plot(xx,yy,'-')
plo.show()
```

W języku c++ można użyć biblioteki MathGL. Może być konieczne zainstalowanie pakietów, które nie są instalowane domyślnie. W przypadku dystrybucji *Fedora* potrzebne są cztery pakiety: mathgl, mathgl-devel, mathgl-qt5 i mathgl-qt5-devel.

Gotowe pakiety są dostępne także dla dystrybucji *Ubuntu*. W razie konieczności kompilacji biblioteki można zciągnąć kod z

http://mathgl.sourceforge.net/doc_en/Download.html#Download

Program tworzący zbiór z wykresem funkcji sinus może mieć postać:

```
#include <mgl2/mgl.h>
#include <iostream>
#include <fstream>
#include <cmath>
using namespace std;

int main(int ,char **){
    mglGraph gr;
    gr.FPlot("sin(pi*x)", "[0,M_PI]");
    gr.Title ("Wykres funkcji sinus");
    gr.WriteJPEG("Wykres_testowy.jpg");
    return 0;
}
```

Przy kompilacji konieczne jest dopisanie opcji -lmgl.

Programik rysujący wykresy w oknie na kranie może wyglądać tak:

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <mgl2/qt.h>
using namespace std;

int ramka(mglGraph *gr){
    mglData y0(50),y1(50),y;
    double *a = new double[50];
    //Poniższe polecenie powoduje powstanie w oknie jednego rysunku.
    gr->SubPlot(1,1,0);
    y0.Modify("sin(pi*(2*x-1))");
    y1.Modify("cos(pi*(2*x-1))");
    //Przykład z tablicą.
    for(int i=0;i<50;i++) a[i] = sin(M_PI*i/49.);
    y.Set(a,50);
    gr->Plot(y0);
    gr->Plot(y1);
    gr->Plot(y);
    gr->Box();
    return 0;
}

int main(int argc,char **argv){
```

```
mg1QT gr(ramka,"Nic");  
gr.Run();  
return 0;  
}
```

Przy kompilacji konieczne jest dopisanie opcji -lmgl -lmgl-qt5.

Rozdział 2

Układy liniowych równań algebraicznych

2.1 Rozwiązywanie układów równań liniowych

Układ n liniowych równań n zmiennych $x_1, \dots, x_n$ można zapisać w postaci wektorowej $\mathbf{A} \mathbf{x} = \mathbf{y}$:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n-1} & a_{1,n} \\ & & \vdots & & \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n-1} & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}. \quad (2.1)$$

Istnieją różne metody rozwiązywania takich równań, w tym *eliminacja Gaussa-Jordana*, *dekompozycja LU*, oraz metody kolejnych poprawek przybliżonego rozwiązania (*metoda Gaussa-Seidla*, *Jacobiego*, *Richardsona*, *sukcesywnej nadrelaksacji SOR*). Metoda eliminacji Gaussa-Jordana jest łatwa do zrozumienia, a napisanie odpowiedniego programu zajmuje niewiele czasu. Ponadto, metoda ta nadaje się nie tylko do rozwiązywania układów równań, ale także do znajdowania macierzy odwrotnej $\mathbf{A}^{-1}$. Metoda dekompozycji LU pozwala dużo szybciej uzyskać rozwiązanie układu równań i obliczyć wyznacznik macierzy, ale nie daje macierzy odwrotnej.

Należy pamiętać, że z powodu błędów numerycznych błąd $\delta \mathbf{x}$ uzyskanego rozwiązania $\mathbf{x}_1 = (\mathbf{x} + \delta \mathbf{x})$ może nie być pomijalny. Dlatego wskazane jest sprawdzenie wyniku. Jeżeli dokładność nie jest satysfakcjonująca można spróbować skorygować rozwiązanie.

Metoda eliminacji Gaussa-Jordana polega ona na przekształceniu równania

do postaci z macierzą trójkątną

$$\begin{bmatrix} a_{1,1}^* & a_{1,2}^* & \cdots & a_{1,n-1}^* & a_{1,n}^* \\ & & \vdots & & \\ 0 & 0 & \cdots & 0 & a_{n,n}^* \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1^* \\ \vdots \\ y_n^* \end{bmatrix}. \quad (2.2)$$

Etap pierwszy polega na zamienieniu na zera wszystkich za wyjątkiem pierwszego elementów pierwszej kolumny macierzy. Kolejność czynności jest następująca:

- zapisanie pierwszego elementu pierwszego wiersza macierzy $a = a_{1,1}$,
- podzielenie współczynników $a_{1,1} \dots a_{1,n}$, oraz y_1 przez a i pomnożenie przez $a_{2,1}$,
- odjęcie uzyskanych w poprzednim kroku $\frac{a_{1,j}}{a}$ od $a_{2,j}$, oraz y_2
- pomnożenie zmodyfikowanych elementów pierwszego wiersza oraz y_1 przez a i podzielenie przez $a_{3,1}$,
- odjęcie uzyskanych w poprzednim kroku współczynników od $a_{3,j}$, oraz y_3
- ...aż do ostatniego wiersza.

Drugi etap jest analogiczny do pierwszego. Zamiast całej macierzy $n \times n$ w postaci uzyskanej po pierwszym etapie poddaje się modyfikacji macierz pomniejszoną o pierwszy wiersz i pierwszą kolumnę.

Procedurę eliminacji można zapisać w postaci

```
for(int i=1; i<=n-1; i++) {
    a=tab[i][i];
    for(int k=i+1; k<=n; k++) {
        bb=tab[k][i];
        for(int j=i; j<=n; j++) {
            tab[k][j]=tab[k][j]-tab[i][j]/aa*bb;
        }
        y[k]=y[k]-y[i]*bb/aa;
    }
}
```

gdzie $\text{tab}[i][j]$ oznacza współczynnik $a_{i,j}$. Ta procedura nie gwarantuje występowania na przekątnej samych jedynek. Nie ma to jednak znaczenia kiedy chodzi tylko o rozwiązanie równania 2.2.

Źródłem niedokładności jest dzielenie przez liczby znajdujące się na przekątnej macierzy. Przy dzieleniu przez liczbę bliską zera pojawia się błąd, który rośnie przy kolejnych operacjach eliminacji. Problem można ograniczyć stosując *pivoting*. Najprościej jest przestawiać wiersze równania w taki sposób, żeby element na przekątnej, przez który będzie wykonywane dzielenie był największy.

Równanie 2.2 można łatwo rozwiązać od końca. Najpierw z ostatniego równania $a_{n,n}^* x_n = y_n^*$ wyznacza się x_n , następnie z przedostatniego $a_{n-1,n-1}^* x_{n-1} + a_{n-1,n}^* x_n = y_{n-1}^*$ oblicza się x_{n-1} i tak dalej.

2.1.1 Korekcja rozwiązania

Podstawienie rozwiązania $\mathbf{x}_1 = (\mathbf{x} + \delta\mathbf{x})$ do równania 2.1 prowadzi do zależności

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) - \mathbf{y} = \delta\mathbf{y}. \quad (2.3)$$

Odejmując równanie 2.1 można uzyskać

$$\mathbf{A}\delta\mathbf{x} = \delta\mathbf{y}, \quad (2.4)$$

czyli równanie na błąd $\delta\mathbf{x}$ poprzednio uzyskanego rozwiązania. Uzyskane $\delta\mathbf{x}_1 \simeq \delta\mathbf{x}$ można podstawić do równania 2.3, co doprowadzi do zależności

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}_1) - \mathbf{y} = \delta\mathbf{y}_1, \quad (2.5)$$

której można użyć do wyznaczenia poprawki kolejnego rzędu. Należy jednak pamiętać, że iteracyjne metody nie zawsze są zbieżne.

2.1.2 Macierz zespolona

W przypadku kiedy współczynniki w układzie równań są liczbami zespolonymi, równanie 2.1 należy zapisać w postaci

$$(\mathbf{A} + i\mathbf{C})(\mathbf{x}_r + i\mathbf{x}_i) = (\mathbf{y}_r + i\mathbf{y}_i), \quad (2.6)$$

z wydzieloną częścią urojoną. Takie równanie można przedstawić jako układ dwu równań wektorowych

$$\begin{aligned} (\mathbf{A}\mathbf{x}_r - \mathbf{C}\mathbf{x}_i) &= \mathbf{y}_r \\ (\mathbf{C}\mathbf{x}_r + \mathbf{A}\mathbf{x}_i) &= \mathbf{y}_i, \end{aligned}$$

albo jednego równania z macierzą blokową $2n \times 2n$ o antysymetrycznie rozmieszczonych blokach. Równanie to ma postać

$$\begin{bmatrix} \mathbf{A} & -\mathbf{C} \\ \mathbf{C} & \mathbf{A} \end{bmatrix} \mathbf{X} = \mathbf{Y}. \quad (2.7)$$

gdzie wektory X i Y są złożone odpowiednio z x_r i x_i , oraz y_r i y_i .

W języku c++ liczby zespolone są dostępne po wpisaniu w nagłówku `#include<complex>`.

Definicja liczby zespolonej jest podobna do definicji liczby rzeczywistej. Jeżeli $z = (a_r, b_r)$ ma mieć podwójną precyzję, należy napisać

`complex<double> z = a_r + b_r i;`

Należy pamiętać, że kiedy $b_r = 1$ część urojoną zapisuje się jako $1i$, a nie i .

2.2 Macierz odwrotna, wyznacznik, wartości własne

Jeżeli macierz $\mathbf{A}$ jest odwracalna, można znaleźć macierz odwrotną $\mathbf{A}^{-1}$ np. przy użyciu eliminacji Gaussa-Jordana. Opisana w poprzednim rozdziale metoda rozwiązywania równania $\mathbf{A} \mathbf{x} = \mathbf{y}$ wymagała jedynie wyeliminowania niezerowych elementów z poniżej przekątnej macierzy $\mathbf{A}$. W analogiczny sposób można wyeliminować niezerowe elementy powyżej przekątnej. W celu wyznaczenia $\mathbf{A}^{-1}$ należy wykonać dwie operacje opisane poniżej.

1. Do macierzy kwadratowej $\mathbf{A}$ trzeba dopisać z prawej strony macierz jednostkową $\mathbf{I}$.
2. W macierzy $\mathbf{AI}$ należy wyeliminować niezerowe elementy od dołu i od góry przekątnej części $\mathbf{A}$. Kiedy na przekątnej części $\mathbf{A}$ są jedynki, a po obu stronach zera, wówczas prawa połowa $\mathbf{AI}$ jest $\mathbf{A}^{-1}$.

Program może mieć postać jak poniżej.

```
//Eliminacja od dołu
for(int i=1;i<=n;i++){
    aa=arr[i][i];
    for(int k=i+1;k<=n;k++){
        bb=arr[k][i];
        for(int j=i;j<=2*n;j++){
            arr[k][j]=arr[k][j]-arr[i][j]*bb/aa;
```

```

        }
    }
}
//Tworzenie jedynek na przekatnej
for(int i=1;i<=n; i++){
    aa=arr[i][i];
    for(int k=1;k<=2*n;k++) arr[i][k]=arr[i][k]/aa;
}
//Eliminacja od gory
for(int i=n;i>=1;i-){
aa=arr[i][i];
    for(int k=i-1;k>=1;k-){
        bb=arr[k][i];
        for(int j=2*n;j>=1;j-) arr[k][j]=arr[k][j]-arr[i][j]*bb;
    }
}
}

```

Wyznacznik macierzy $\mathbf{A}$ można obliczyć stosując rozkład Laplace'a

$$\det(\mathbf{A}) = a_{1,1} \begin{vmatrix} a_{2,2} & \cdots & a_{2,n} \\ \vdots & & \\ a_{n,2} & \cdots & a_{n,n} \end{vmatrix} - a_{1,2} \begin{vmatrix} a_{2,1} & a_{2,3} & \cdots & a_{2,n} \\ \vdots & & & \\ a_{n,1} & a_{n,3} & \cdots & a_{n,n} \end{vmatrix} + \dots \quad (2.8)$$

Znaki $+$ i $(-)$ występują naprzemiennie.

Ćwiczenie. Proszę napisać procedurę rozwiązującą równania typu 2.1 i wyznaczający macierz odwrotną do macierzy $\mathbf{A}$. Procedura powinna mieć argumenty: $\mathbf{A}$, $\mathbf{x}$, $\mathbf{y}$, n . Można zastosować dowolną metodę. Procedurę należy przetestować przy użyciu równań:

$$\begin{bmatrix} 0.1 & 1 & 1 & 1 \\ 1 & 0.1 & 1 & 1 \\ 1 & 1 & 0.1 & 1 \\ 1 & 1 & 1 & 0.1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}, \quad (2.9)$$

oraz

$$\begin{bmatrix} 10^{-15} & 5 & 5 & 5 \\ 5 & 10^{-15} & 5 & 5 \\ 5 & 5 & 10^{-15} & 5 \\ 5 & 5 & 5 & 10^{-15} \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}. \quad (2.10)$$

Proszę sprawdzić dokładność uzyskanych wyników. W przypadku kiedy metoda nieiteracyjna daje błąd przekraczający 1% należy zastosować korekcję.

Wskazówka. W języku C++ można zadeklarować bez podania rozmiaru tablicę jednowymiarową, ale nie macierz. W związku z tym można przed wywołaniem procedury rozkładać macierz na tablicę o długości $n \times n$, a w procedurze składać na powrót do postaci macierzy.

2.2.1 Wartości własne i wektory własne

Wartości własne i wektory własne macierzy $\mathbf{A}$ spełniają równania

$$\det |\mathbf{A} - a_i \mathbf{I}| = 0, \quad (2.11)$$

oraz

$$\mathbf{A} \mathbf{x}_i = a_i \mathbf{x}_i. \quad (2.12)$$

Można więc łatwo sprawdzić czy zadana liczba jest wartością własną, ale niestety nie jest łatwo znaleźć nieznane wartości własne i odpowiadające im wektory własne.

Można najpierw rozwiązać równanie 2.11 i w ten sposób uzyskać wartości własne a_i , a następnie spróbować odgadnąć rozwiązanie równania

$$|\mathbf{A} - a_i \mathbf{I}| \mathbf{x}_i = 0. \quad (2.13)$$

Leiej jednak zastosować inną metodę.

Stosunkowo prostą koncepcyjnie, ale powolną metodą wyznaczania wartości i wektorów własnych jest *odwrotna iteracja*. Poniżej są opisane kolejne etapy.

1. Trzeba wybrać wartość $a_{i,0}$ zbliżoną do i -tej wartości własnej i wygenerowanie dowolnego wektora $\mathbf{P}_{i,0}$, na przykład przy użyciu generatora liczb pseudolosowych.
2. Następnie trzeba rozwiązać równanie

$$(\mathbf{A} - a_{i,0} \mathbf{I}) \mathbf{x}_{i,0} = \mathbf{P}_{i,0}, \quad (2.14)$$

żeby wyznaczyć pierwsze przybliżenie $\mathbf{x}_{i,0}$ i -tego wektora własnego.

3. Na podstawie wyznaczonego $\mathbf{x}_{i,0}$ trzeba wyznaczyć

$$\mathbf{P}_{i,1} = \frac{\mathbf{x}_{i,0}}{|\mathbf{x}_{i,0}|}, \quad (2.15)$$

oraz

$$a_{i,1} = a_{i,0} + \frac{1}{\mathbf{P}_{i,0} \cdot \mathbf{x}_{i,0}}. \quad (2.16)$$

4. Mając $a_{i,1}$ i $\mathbf{P}_{i,1}$ należy wrócić do punktu 2, czyli w równaniu 2.14 zastąpić poprzednie przybliżenia nowymi i wyznaczyć nowe przybliżenie wektora $\mathbf{x}_i$.

Jako kryterium osiągnięcia satysfakcjonującej dokładności dobrze jest przyjąć spadek szybkości zmian a , np. $\text{abs}(\frac{1}{\mathbf{P}_{i,1} \mathbf{x}_{i,1}}) < 0.0001$.

Przykład

Jeżeli macierz $\mathbf{A}$ ma postać

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad (2.17)$$

wówczas można przyjąć na początek $a_{1,0} = -0.4$, oraz

$$\mathbf{P}_{i,0} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad (2.18)$$

Po $l = 4$ krokach procedura iteracji prowadzi do $a_{1,l} = -0.372281$ i wektora

$$\mathbf{P}_{1,1} = x \begin{bmatrix} -1.45743 \\ 1 \end{bmatrix}, \quad (2.19)$$

Jeżeli $a_{2,0} = +0.4$, wówczas $a_{2,5} = 5.37229$.

Rozdział 3

Rozwiązywanie równań nieliniowych o pierwiastkach rzeczywistych

3.1 Metoda bisekcji

Metoda bisekcji jest prosta i ma wysoką niezawodność, ale jest powolna. Pierwszym krokiem jest określenie wymaganej dokładności. Można zdefiniować ϵ takie, że w pobliżu pierwiastka $|f(x)| < \epsilon$, albo ϵ_x dla którego $|x_a - x| < \epsilon_x$. Następnie trzeba wybrać dwa punkty x_a i x_b dla których jest spełniony warunek $f(x_a)f(x_b) < 0$. W przypadku niewłaściwego wyboru zamiast pierwiastka może zostać znaleziona osobliwość, jeżeli istnieje. Następne kroki są następujące:

1. Wyznaczenie punktu środkowego $x_c = 0.5(x_a + x_b)$.
2. Sprawdzenie, czy x_c jest dostatecznie bliskie pierwiastka i ewentualne zakończenie obliczeń.
3. Sprawdzenie w którym przedziale znajduje się pierwiastek i zawężenie badanego zakresu. Jeżeli $f(x_a)f(x_c) < 0$ wówczas $x_c \rightarrow x_b$, natomiast kiedy $f(x_c)f(x_b) < 0$ wówczas $x_c \rightarrow x_a$.
4. Powrót do punktu 1.

Przykładowy program znajdujący zero i mierzący czas obliczeń:

```
#include <iostream>
#include <cmath>
#include <fstream>
#include <ctime>
#include <iomanip>
using namespace std;
```

```

double f(double xx){
double a,b,c;
a=xx*xx-4.0;
return a;
}
int main(){
double a,b,c,d,w,dw,epsilon;
cout << "Granice przedzialu calkowania (a,b):";
cin >> a >> b;
cout << a << "<< b << endl;
epsilon = 0.00000000001;
double_t pocz = clock();
do{
if(f(a)*f(b) > 0.){cout << "Brak zera"<<endl; return 0;}
c = (a+b)/2.0;
if( f(a)*f(c) < 0.0 )b=c;
if( f(c)*f(b) < 0.0 )a=c;
}
while( abs(f(c)) > epsilon );
double czas = (clock() - pocz)/double(CLOCKS_PER_SEC);
cout << " c = "<< setprecision(15) << c << endl;
cout << "Czas obliczen: "<<czas<< " s"<<endl;
return 0;
}

```

3.2 Metoda Newtona-Raphsona

Ta metoda szybko daje wynik jeżeli wartość bezwzględna pochodnej funkcji maleje przy zbliżaniu się do pierwiastka. Na początku trzeba wyznaczyć tylko jeden punkt x_0 leżącego w pobliżu szukanego pierwiastka. Kolejne przybliżenia oblicza się ze wzoru

$$x_1 = x_0 - f(x_0) \left(\frac{df}{dx}(x_0) \right)^{-1}. \quad (3.1)$$

Pierwszą pochodną funkcji $f(x)$ można wyznaczyć przy użyciu wzoru

$$\frac{df}{dx} \sim \frac{f(x+h) - f(x-h)}{2h}. \quad (3.2)$$

W przypadku drugiej pochodnej dobrze sprawdza się przybliżenie

$$\frac{d^2 f}{dx^2} \sim \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}. \quad (3.3)$$

3.3 Metoda falsi

Podobnie jak w metodzie bisekcji trzeba znaleźć punkty x_a i x_b ograniczające zakres w którym szukany jest pierwiastek. Jednak zamiast znajdowania punktu środkowego należy wyznaczyć pierwsze przybliżenie jako punkt x_f w którym prosta przechodząca przez punkty $(x_a, f(x_a))$ i $(x_b, f(x_b))$ przecina oś X. Następnie, podobnie jak w metodzie bisekcji trzeba sprawdzić po której stronie przybliżonego rozwiązania leży pierwiastek ($f(x_a)f(x_f) < 0$, czy $f(x_f)f(x_b) < 0$) i odpowiednio zawęzić badany przedział.

3.4 Metoda Riddersa

Jest to metoda oparta na metodzie falsi, wymagająca mniejszej liczby iteracji. Po znalezieniu punktu $x_c = \frac{1}{2}(x_a + x_b)$. = jak w metodzie bisekcji szukamy funkcji wykładniczej e^u takiej, żeby pasowała do analizowanej funkcji f w x_a , x_c i x_b , czyli

$$f(x_a) - 2f(x_c)e^u + f(x_b)e^{2u} = 0. \quad (3.4)$$

Ten warunek jest spełniony jeżeli

$$e^u = \frac{1}{f(x_b)} \left(f(x_c) + \text{sign}(f(x_a)) \sqrt{f(x_c)^2 - f(x_a)f(x_b)} \right). \quad (3.5)$$

Po zastosowaniu metody falsi do punktów $(x_a, f(x_a)e^u)$ i $(x_b, f(x_b)e^{2u})$ (zamiast $(x_a, f(x_a))$ i $(x_b, f(x_b))$) dostaje się kolejne przybliżenie szukanego pierwiastka:

$$x_d = x_c + (x_c - x_a) \frac{\text{sign}(f(x_a) - f(x_c))f(x_c)}{\sqrt{f(x_c)^2 - f(x_a)f(x_b)}}. \quad (3.6)$$

Następnie należy rozważyć trzy przedziały (a - d, d - c, c - b) i sprawdzić w którym z nich jest zero funkcji:

$$f(x_a)f(x_d) < 0 \Rightarrow x_b \leftarrow x_d$$

$$f(x_d)f(x_b) < 0 \Rightarrow x_a \leftarrow x_d$$

$f(x_d)f(x_c) < 0 \Rightarrow x_a \leftarrow x_c, x_b \leftarrow x_d$
 Dla właściwego przedziału należy obliczyć

$$x_c = \frac{1}{2}(x_a + x_b), \quad (3.7)$$

a następnie przejść do następnej iteracji, czyli wyznaczyć punkt x_d z równania 3.6.

Ćwiczenie

Zaimplementować metodę różniczkowania numerycznego za pomocą różnic dzielonych

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x)}{h} \quad (3.8)$$

gdzie parametrem jest długość przedziału h .

- Obliczyć pochodną funkcji $\sin(x)$ w przedziale $[0, 2\pi]$ i porównać graficznie z wynikiem dokładnym.
- Dla punktu $\pi/4$ sprawdzić jak zmienia się dokładność wraz ze zmianą parametru h .
- Powtórzyć analizę dla funkcji

$$f(x) = \frac{-V}{1 + e^{(x-x_0)/a}} \quad (3.9)$$

Dla $V=1, x_0=1, a=0.1$, w punkcie $x=1.0$.

Ćwiczenie

Zaimplementować metodę różniczkowania numerycznego za pomocą przybliżenia 3.2. Przeprowadzić analizę jak w poprzednim zadaniu oraz porównać graficznie rezultaty obu metod w punkcie b .

Ćwiczenie. Proszę znaleźć zero funkcji $f(x) = x^2 - 4$ oraz $f(x) = \tanh(x)$. W obu przypadkach należy szukać zera w przedziale od -1 do $+10$. Która metoda jest najszybsza?

Rozdział 4

Interpolacja

Interpolacja polega na znajdowaniu krzywej dokładnie przechodzącej przez zadane punkty, zwykle stanowiące wyniki pomiarów. Najprościej jest znaleźć równania odcinków łączących kolejne pary punktów. Jednak wówczas nie uzyskamy gładkiej krzywej.

4.1 Wielomian Lagrange

Przykładem krzywej gładkiej przechodzącej przez n punktów jest wielomian Lagrange stopnia $n - 1$.

4.1.1 Metoda 1

Jeżeli zmienną niezależną oznaczmy przez t (np. czas), a zmienną mierzoną przez T (np. temperatura), wówczas krzywa ma przechodzić przez punkty (t_i, T_i) . Wartości wielomianu dla dowolnego t takiego że $t_1 < t < t_n$ można łatwo obliczyć np. ze wzoru

$$T(t) = \sum_{i=1}^{i=n} T_i \prod_{j \neq i} \frac{t - t_j}{t_i - t_j}. \quad (4.1)$$

Główna część programu może być:

```
int main() {  
    ofstream ofile;  
    ifstream ifile;  
    int n,mm;  
    int i,j;  
    double x,dx,yy,ilocz,aa;
```

```

ofile.open("interpolacja_wyniki_rownanie4-1.txt");
ifile.open("interpolacja_wejscowe.txt");
ifile >> n;
mm=(n-1)*10;
double d[n+1][n+1], xw[n+1], yw[n+1];
for(int kk=1; kk<=n; kk++){
    ifile >> xw[kk] >> yw[kk];
}
dx=(xw[n]-xw[1])/float(mm);
x=xw[1];
do {
    yy=0.0;
    for (int i=1;i<=n;i++){
        ilocz=1.0;
        for (int j=1;j<i;j++){
            aa=(x-xw[j])/(xw[i]-xw[j]);
            ilocz=ilocz*aa;
        }
        for (int j=i+1;j<=n;j++){
            aa=(x-xw[j])/(xw[i]-xw[j]);
            ilocz=ilocz*aa;
        }
        yy=yy+yw[i]*ilocz;
    }
    x=x+dx;
    cout << x << ' ' << yy << endl;
    ofile << x << ' ' << yy << endl;
}
while(x<xw[n]);
ofile.close();
ifile.close();
}

```

4.1.2 Metoda 2

Można też wykonać obliczenia metodą kolejnych poprawek (algorytm Neville). Języki programowania mogą traktować małe i duże litery tak samo, więc dobrze wprowadzić tablice o dwuliterowych nazwach:

$tx[n]$ dla $t_0, \dots, t_{n-1}$, oraz
 $ty[n]$ dla $T_0, \dots, T_{n-1}$.

Zerową (o numerze $m = 0$) serią przybliżeń są szukanego $T(t)$ są wartości mierzone T_i , których jest n . Można je wpisać do zerowego (albo pierwszego, jeżeli numeracja jest od 0) wiersza macierzy $d[n,n]$:

```
for (int i=0; i<n; i++) d[0][i]=ty[i]
```

Następnie trzeba znaleźć proste przechodzące przez pary punktów $(tx[i], ty[i])$, jest ich już tylko $n - 1$. Te proste służą do wyznaczenia następnej (o numerze $m=1$) serii przybliżeń szukanego $T(t)$. W każdym kolejnym kroku jest o jedno przybliżenie mniej, a w ostatnim już tylko jedno. To właśnie szukana wartość $T(t)$ Odpowiedni fragment programu może wyglądać tak:

```
for (int m=1; m<n; m++) {
    for (int i=1; i<n-m; i++) {
        d[m][i]=
            ((t-tx[i+m])*d[m-1][i]-(t-tx[i])*d[m-1][i+1])/
            (tx[i]-tx[i+m]);
        yy=d[m][i];
    }
}
```

W tym programie numerowanie jest od 1. Przy numerowaniu od zera trzeba wprowadzić zmianę.

Niestety, wynik interpolacji nie zawsze jest zadowalający. Warto wkonać interpolację dla zestawu punktów o współrzędnych przedstawionych w tabelicy 4.1. Leżą na krzywej ze schodkiem. W takich przypadkach koło skoku

Tabela 4.1: Współrzędne punktów.

t: 0	1	2	3	4	5	6	7	8	9
T: 0	0	0	0	0	10	10	10	10	10

generują się fale. Im wyraźniejszy skok, tym gorzej. Krzywa opisana przez wielomian Lagrangea stopnia $n-1$ ma w tym przypadku dodatkową wadę. Na końcach zakresu wartości bezwzględne pierwszej pochodnej są nierealistycznie duże.

Ćwiczenie. Proszę znaleźć wartości wielomianu interpolacyjnego dla punktów z tablicy 4.1 i umieścić na jednym wykresie zarówno zadane punkty, jak i obliczone wartości. Obliczenia należy wykonać dla minimum 100 punktów. Warto porównać czas potrzebny przy użyciu równania 4.1 i przy zastosowaniu metody iteracyjnej.

4.2 Cubic spline

Dobry efekt daje interpolacja 'cubic spline'. Zapewnia ona zarówno gładkość krzywej (ciągłość pierwszej pochodnej), jak i brak niepożądanych fluktuacji przy końcach zakresu. Koncepcja polega na tym, żeby najpierw zrobić interpolację odcinkową, a potem ją poprawić. Dla każdej pary punktów $(i, i + 1)$ do równania odcinka dodaje się równanie wielomianu w 3 stopnia, który ma wartość zero w punktach i , oraz $i + 1$. Wówczas krzywa łącząca punkty i oraz $i + 1$ jest dana równaniami

$$\begin{aligned} T &= AT_i + (1 - A)T_{i+1} + CT_i'' + DT_{i+1}'', \\ A &= (t_{i+1} - t)/(t_{i+1} - t_i), \\ C &= \frac{1}{6}(A^3 - A)(t_{i+1} - t_i)^2 \\ D &= \frac{1}{6}(B^3 - B)(t_{i+1} - t_i)^2, \end{aligned} \quad (4.2)$$

gdzie symbole T_i'' i T_{i+1}'' oznaczają pochodne $\frac{d^2T}{dt^2}$ w t_i i t_{i+1} . Łatwo zauważyć, że kiedy $C = D = 0$ równanie 4.2 opisuje prostą. Równanie 4.2 nie spełnia jeszcze warunku żeby w w każdym punkcie i pochodne wielomianu w obliczone dla punktów $(i - 1, i)$, oraz $(i, i + 1)$ były takie same. Warunek ten jest spełniony kiedy drugie pochodne zmiennej T są zadane równaniem

$$\begin{aligned} \frac{1}{6}(t_i - t_{i-1})T_{i-1}'' + \frac{1}{3}(t_{i+1} - t_{i-1})T_i'' + \frac{1}{6}(t_{i+1} - t_i)T_{i+1}'' = \\ \frac{T_{i+1} - T_i}{t_{i+1} - t_i} - \frac{T_i - T_{i-1}}{t_i - t_{i-1}}. \end{aligned} \quad (4.3)$$

Rozwiązanie układu n równań dla n punktów $i = 0, \dots, n - 1$ wymaga dodania (t_{-1}, T_{-1}) , oraz (t_n, T_n) . Punkty te powinny zapewniać zerową wartość pochodnej $\frac{dT}{dt}$ dla $t = t_0$ i t_{n-1} . Jest tak kiedy

$$\begin{aligned} (t_{-1}, T_{-1}) &= (2t_0 - t_1, T_1), \\ (t_n, T_n) &= (2t_{n-1} - t_{n-2}, T_{n-2}). \end{aligned} \quad (4.4)$$

Podstawienie tych zależności do równania 4.3 dla $i = 0$, oraz $i = n - 1$ daje odpowiednio

$$\begin{aligned} \frac{1}{6}(t_1 - t_0)T_1'' + \\ \frac{1}{3}(t_1 - (2t_0 - t_1))T_0'' + \\ \frac{1}{6}(t_1 - t_0)T_1'' = \\ \frac{T_1 - T_0}{t_1 - t_0} - \frac{T_0 - T_1}{t_0 - (2t_0 - t_1)}, \end{aligned} \quad (4.5)$$

oraz

$$\begin{aligned}
& \frac{1}{6}(t_{n-1} - t_{n-2})T''_{n-2} + \\
& \frac{1}{3}((2t_{n-1} - t_{n-2}) - t_{n-2})T''_{n-1} + \\
& \frac{1}{6}((2t_{n-1} - t_{n-2}) - t_{n-1})T''_{n-2} = \\
& \frac{T_{n-2} - T_{n-1}}{(2t_{n-1} - t_{n-2}) - t_{n-1}} - \frac{T_{n-1} - T_{n-2}}{t_{n-1} - t_{n-2}}.
\end{aligned} \tag{4.6}$$

Równania 4.5 i 4.6 można przekształcić do postaci

$$\frac{2}{3}(t_1 - t_0)T''_0 + \frac{2}{6}(t_1 - t_0)T''_1 = 2\frac{T_1 - T_0}{t_1 - t_0}, \tag{4.7}$$

$$\frac{2}{6}(t_{n-1} - t_{n-2})T''_{n-2} + \frac{2}{3}(t_{n-1} - t_{n-2})T''_{n-1} = -2\frac{T_{n-1} - T_{n-2}}{t_{n-1} - t_{n-2}}. \tag{4.8}$$

Ostatecznie, wykonanie interpolacji dla n punktów (t_i, T_i) , gdzie $i = 0, \dots, n-1$ wymaga rozwiązania układu n równań: równanie 4.7 dla $i=0$, równanie 4.3 dla $0 < i < n-1$, oraz równanie 4.8 dla $i = n-1$. W zapisie wektorowym jest to równanie z macierzą trójkątną, a więc do jego rozwiązania można użyć specjalnego algorytmu. Można jednak posłużyć się uniwersalnym algorytmem np. eliminacją Gaussa-Jordana (rozdział 2.1).

Ćwiczenie. Proszę wykonać interpolację cubic-spline dla zestawu punktów z tablicy 4.1.

Rozdział 5

Całkowanie

Całki wielu funkcji można policzyć analitycznie, ale nie zawsze jest to proste. Natomiast zapisanie programu obliczającego przybliżoną wartość całki zwykle nie jest trudne. Problem występuje np. kiedy nie można obliczyć wartości funkcji w granicy zakresu całkowania, choć całka istnieje, kiedy granica nie jest liczbą skończoną, albo kiedy w zakresie całkowania istnieje całkowalna osobliwość.

5.1 Metoda Newtona-Cotesa

Kiedy trzeba obliczyć całkę funkcji $f(x)$ w zakresie $x_p - x_k$ najprościej jest użyć przybliżenia

$$\int_{x_p}^{x_k} f(x) dx \simeq I_n \equiv \Delta_x \left[\frac{1}{2}(f(x_p) + f(x_k)) + \sum_{i=1}^{n-1} f(x_p + \Delta_x i) \right], \quad (5.1)$$

gdzie $\Delta_x = (x_k - x_p)/(n - 1)$. Im większe jest n , tym lepsze jest przybliżenie. Jednak przy arbitralnym założeniu jakiejś wartości n nie można określić dokładności przybliżenia. W związku z tym należy wykonać serię prób, jak poniżej.

1. Przyjąć satysfakcjonującą dokładność np. 1% (stosunek błędu do wartości całki $\epsilon = 0.01$).
2. Obliczyć przybliżoną wartość całki I_{n_0} i I_{n_1} , gdzie $n_1 = 2(n_0 - 1) + 1$.
3. Obliczyć względną różnicę

$$\Delta = \left| \frac{I_{n_1} - I_{n_0}}{I_{n_1}} \right| \quad (5.2)$$

4. Jeżeli $\Delta > \epsilon$ podstawić $n_1 \rightarrow n_0$ i $I_{n_1} \rightarrow I_{n_0}$, a następnie obliczyć nowe $n_1 = 2(n_0 - 1) + 1$ i nowe I_{n_1} . Wrócić do punktu 2.

Spełnienie warunku $\Delta \leq \epsilon$ oznacza, że rząd wielkości błąd powinien być taki jak ϵ . W celu dokładniejszego określenia wartości całki można wyznaczyć granicę $\lim_{\Delta_x \rightarrow 0} I$.

5.2 Metoda Simpsona

Zamiast równania 5.1 można użyć

$$\int_{x_p}^{x_k} f(x) dx \simeq \Delta_x \frac{1}{3} (f(x_p) + f(x_k)) + \Delta_x \left[\frac{4}{3} f(x_p + \Delta_x) + \frac{2}{3} f(x_p + 2\Delta_x) + \frac{4}{3} f(x_p + 3\Delta_x) + \dots + \frac{2}{3} f(x_k - 2\Delta_x) + \frac{4}{3} f(x_k - \Delta_x) \right], \quad (5.3)$$

5.3 Kwadratury Gaussa

W tym przypadku nie potrzeba dużej liczby obliczeń, ale za to trzeba pamiętać dokładne wartości parametrów. Wartość całki jest przybliżana przez sumę iloczynów wartości funkcji i wag

$$\int_{x_p}^{x_k} W(x) f(x) dx \simeq \sum_{i=1}^{i=n} w_i f(x_i), \quad (5.4)$$

ale w odróżnieniu od poprzednich metod punkty nie są równoodległe. Funkcja $W(x)$ zależy od wariantu metody.

W przypadku kwadratury Gaussa-Legendre $W(x) \equiv 1$, x_i są miejscami zerowymi wielomianu Legendre, natomiast wagi oblicza się przy użyciu równania

$$w_i = \frac{2}{(1 - x_i)^2 (P'_n(x_i))^2}, \quad (5.5)$$

gdzie P_n jest wielomianem Legendre stopnia n . Jeżeli $x_p = -1$ i $x_k = 1$, wówczas wystarczy podstawić do równania 5.4 wartości x_i i w_i jak poniżej.

- Dla $n = 2$ (kwadratura 2 rzędu) $x_1 = -\frac{1}{\sqrt{3}}$, $x_2 = \frac{1}{\sqrt{3}}$, oraz $w_1 = w_2 = 1$.
- Dla $n = 5$ i dla $n = 10$ wartości są podane w tablicy 5.1.

Tabela 5.1: Wartości parametrów dla kwadratury Gaussa-Legendre przy 5 i 10 punktach.

i	x_i	w_i
1	$-\frac{1}{3} \left(5 + 2\sqrt{10/7} \right)^{0.5}$	$\frac{322-13\sqrt{70}}{900}$
2	$-\frac{1}{3} \left(5 - 2\sqrt{10/7} \right)^{0.5}$	$\frac{322+13\sqrt{70}}{900}$
3	0	128/225
4	$\frac{1}{3} \left(5 - 2\sqrt{10/7} \right)^{0.5}$	$\frac{322+13\sqrt{70}}{900}$
5	$\frac{1}{3} \left(5 + 2\sqrt{10/7} \right)^{0.5}$	$\frac{322-13\sqrt{70}}{900}$
1	-0.9739065285171717	0.06667134430868814
2	-0.8650633666889845	0.1494513491505806
3	-0.6794095682990244	0.219086362515982
4	-0.4333953941292472	0.2692667193099964
5	-0.1488743389816312	0.2955242247147529
6	0.1488743389816312	0.2955242247147529
7	0.4333953941292472	0.2692667193099964
8	0.6794095682990244	0.219086362515982
9	0.8650633666889845	0.1494513491505806
10	0.9739065285171717	0.06667134430868814

W przypadku innego zakresu całkowania należy najpierw wykonać podstawienie

$$x_i \rightarrow x_i \frac{1}{2}(x_k - x_p) + \frac{1}{2}(x_p + x_k), \quad (5.6)$$

a potem obliczyć całkę na podstawie równania 5.4 i pomnożyć wynik przez $(x_k - x_p)/2$.

Dokładność całki zależy od dokładności wyznaczenia współczynników. Można także użyć innej liczby punktów, ale wymaga to obliczenia nowego zestawu x_i i w_i dla odpowiedniego stopnia wielomianu. Odpowiedni algorytm można znaleźć np. w Press et al. (2003).

Całka funkcji $\sin(x)$ od 0 do $\pi/2$ wynosi:
1.000008121555498 dla $n = 3$
1.000000000039565 dla $n = 5$
0.9999999999999999 dla $n = 10$.

Rozdział 6

Losowanie

Prostą metodą generowania liczb pseudolosowych jest użycie formuły

$$x_{n+1} = (ax_n + b) \text{ modulo } c. \quad (6.1)$$

Jest to tak zwany generator kongruentny. W języku c++ można napisać `x[i+1] = (a * x[i] + b) % c`.

Wartość pierwszej liczby x_0 , oraz parametrów a, b, c trzeba dobrać tak żeby zminimalizować powtarzalność wyników. Można przyjąć $x_0 = 1$, $a = 16807$, $b = 1$ i $c = 2^{31} - 1$. Generator będzie wówczas zwracać liczby z zakresu od 0 do $x_{max} = 2^{31} - 2 = 2147483646$, więc dobrze jest je znormalizować do jedynki.

Niestety, wystarczy poznać sekwencję 6 kolejnych wygenerowanych liczb aby wyznaczyć wszystkie parametry generatora. W przypadku generatora Merse-
ne-Twister (Matsumoto i Nishimura, 1997) potrzeba sekwencji 624 liczb. Dobry generator powinien zapewniać płaski rozkład prawdopodobieństwa. Można to sprawdzić dzieląc przedział 0 - 1 na równe podprzedziały i sprawdzając ile wygenerowanych liczb znajduje się w każdym z podprzedziałów. Jeżeli jest n_p podprzedziałów i zostało wygenerowanych n liczb, to w każdym podprzedziale powinno być $n_{ocz} = n/n_p$ liczb. Jakość rozkładu można sprawdzić przy pomocy testu χ^2 . Przejście tego testu jest warunkiem koniecznym, ale nie dostatecznym. Dlatego przed użyciem generatora liczb pseudolosowych do celów naukowych należy wykonać dodatkowe testy, na przykład analizę spektralną. Dokładne omówienie różnych generatorów można znaleźć w Park and Miller (1988); L'Ecuyer and Simard (2007); Baldanzi et al. (2020).

Ćwiczenie. Proszę zbadać przy użyciu testu χ^2 napisany przez siebie generator liczb pseudolosowych i obliczyć metodą Monte Carlo wartość liczby π .

Rozdział 7

Transformata Fouriera

Jeżeli y jest ciągłą funkcją czasu, wówczas jej transformata Fouriera ciągłej jest dana wzorem

$$Y(f) = \int_{-\infty}^{\infty} y(t)e^{2\pi i f t} dt, \quad (7.1)$$

gdzie f oznacza częstotliwość.

Zazwyczaj trzeba znaleźć transformatę funkcji y znając jedynie jej wartości w konkretnych punktach. Najlepiej kiedy punkty są równoodległe, ich liczba n parzysta, a amplituda niezerowa tylko w ograniczonym zakresie częstotliwości $|f| < f_{Nyquist} = 1/(2\Delta t)$. Jeżeli zostaną wzięte pod uwagę jedynie częstotliwości $f_j = j/(n\Delta t)$, gdzie $j = -n/2, \dots, n/2$, wówczas

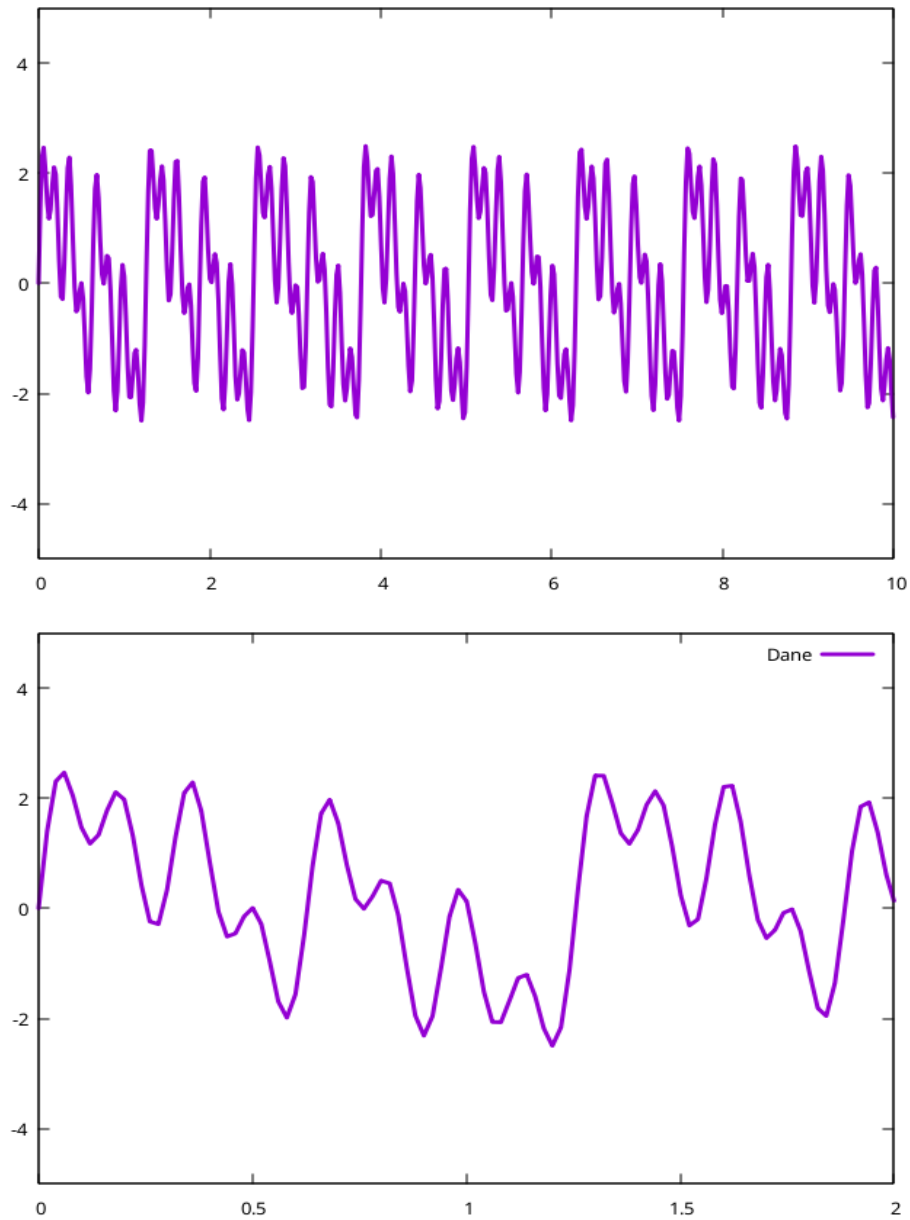
$$Y(f_j) \approx \Delta t \sum_{l=0}^{l=n-1} y_l e^{2\pi i \frac{j}{n\Delta t} l\Delta t} = \Delta t \sum_{l=0}^{l=n-1} y_l e^{2\pi i \frac{j l}{n}} = \Delta t \sum_{l=0}^{l=n-1} y_l S^{jl} = \Delta t Y_j, \quad (7.2)$$

gdzie $t_l = l\Delta t$, $y_l = y(t_l)$, oraz $S = e^{\frac{2\pi i}{n}}$. W sumie trzeba n razy obliczyć sumę n elementów. Jeżeli badana funkcja jest sygnałem akustycznym przepuszczonym przez filtr dolnoprzepustowy o progu 50 kHz próbkowanym 10^5 razy na sekundę przez dziesięć sekund, wówczas trzeba zsumować $n^2 = 10^{12}$ elementów.

Na rysunku 7.1 jest pokazany wykres przykładowej funkcji $y(t)$ o wartościach rzeczywistych

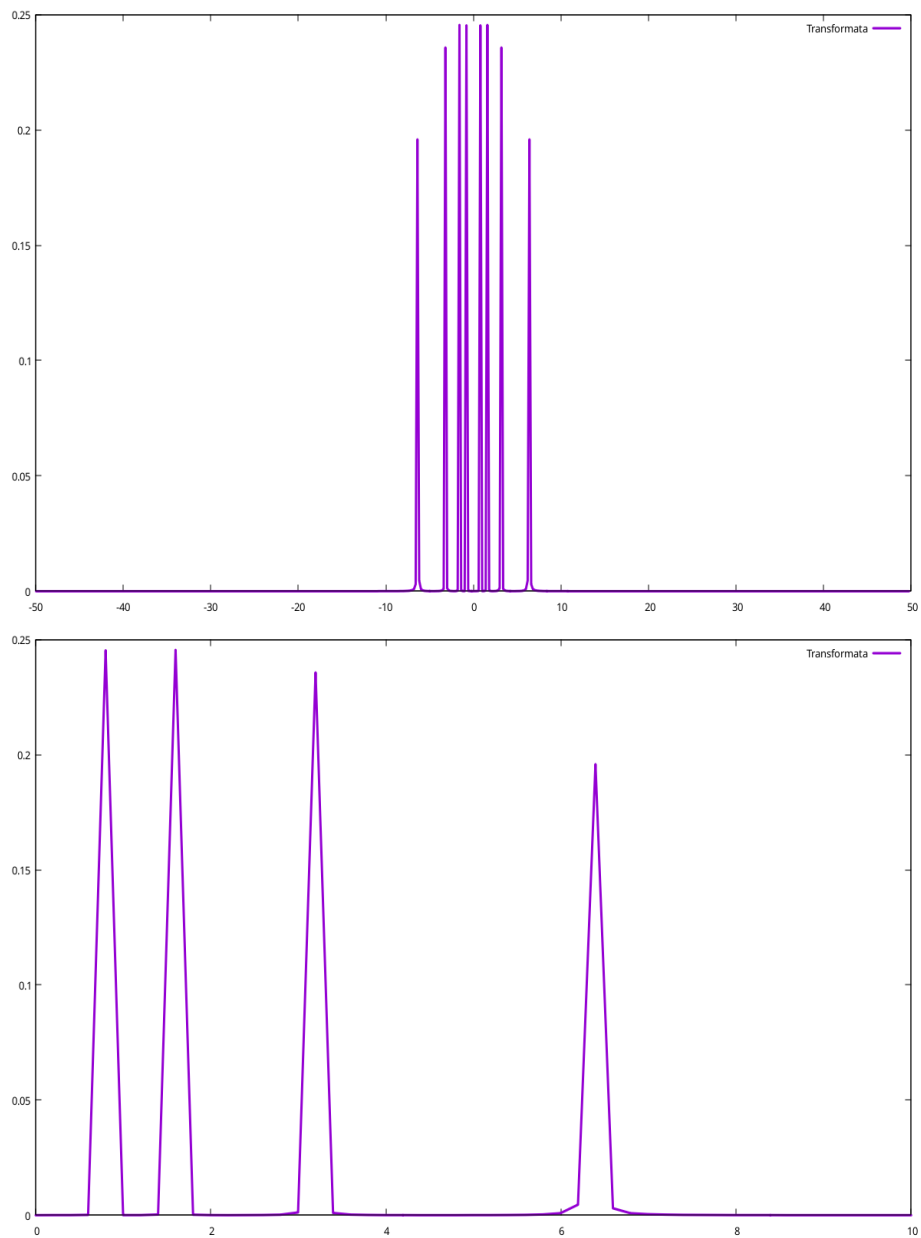
$$y(t) = \sin\left(\frac{2\pi}{\Omega_1}t\right) + \sin\left(\frac{2\pi}{\Omega_2}t\right) + \sin\left(\frac{2\pi}{\Omega_3}t\right) + \sin\left(\frac{2\pi}{\Omega_4}t\right), \quad (7.3)$$
$$\Omega_1 = 2\pi \cdot 0.200, \quad \Omega_2 = 2\pi \cdot 0.100, \quad \Omega_3 = 2\pi \cdot 0.050, \quad \Omega_4 = 2\pi \cdot 0.025,$$

oraz jej początkowy fragment. Próbkowanie zostało wykonane w odstępach $dt = 0.01$ s. Na rysunku 7.2 pokazana jest znormalizowana połowa trans-



Rysunek 7.1: Przykładowa funkcja $y(t)$ o wartościach rzeczywistych.

formaty $Y(f)$ (równanie 7.3, rysunek 7.1), odpowiadająca dodatnim częstotnościom. Normalizacja transformaty oznacza obliczenie $Re[(abs(Y))^2]$.



Rysunek 7.2: Połowa transformaty $Y(f)$ funkcji $y(t)$ pokazanej na rysunku 7.1 (górny panel), oraz powiększenie fragmentu transformaty (dolny panel).

Transformata szybka (FFT) (nie jest wymagane)

Jednak transformata może zostać zapisana jako suma dwu transformat o

połówek długościach:

$$Y_j = \sum_{l=0}^{l=n/2-1} y_{2l} S^{\frac{j}{2}} + S^j \sum_{l=0}^{l=n/2-1} y_{2l+1} S^{\frac{j}{2}}. \quad (7.4)$$

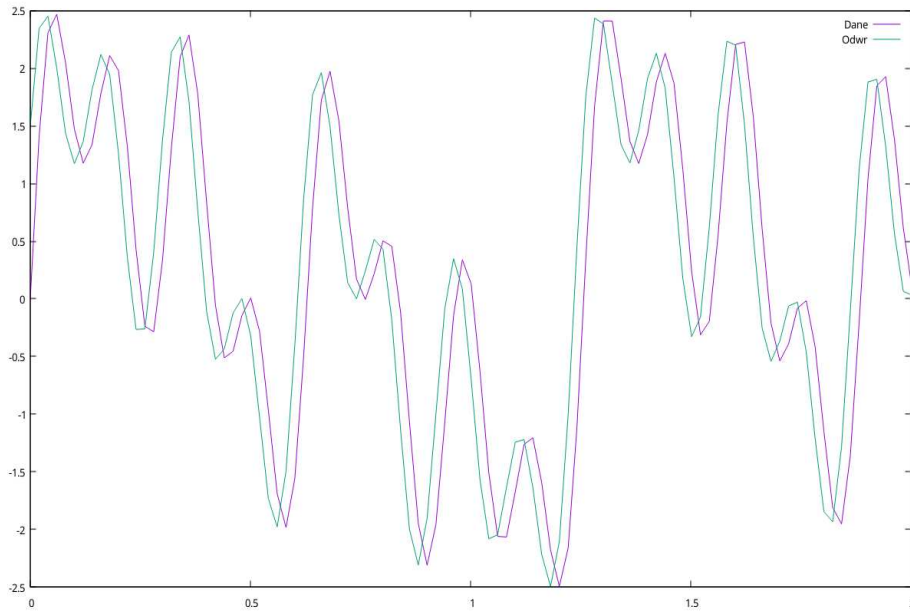
Operację podziału można powtarzać rekurencyjnie.

7.1 Transformata odwrotna, splot

Procedura do obliczania odwrotnej transformaty różni się tylko nieznacznie:

$$y_j \approx \text{Re} \left[\frac{1}{\Delta t n} \sum_{l=0}^{l=n} Y_l e^{-2\pi i \frac{(j-n/2)l}{n}} \right]. \quad (7.5)$$

Na rysunku 7.3 jest pokazane zestawienie fragmentu przebiegu wejściowego (równanie 7.3, rysunek 7.1) i odwrotnej transformaty.



Rysunek 7.3: Porównanie fragmentu przebiegu wejściowego i tr. odwrotnej.

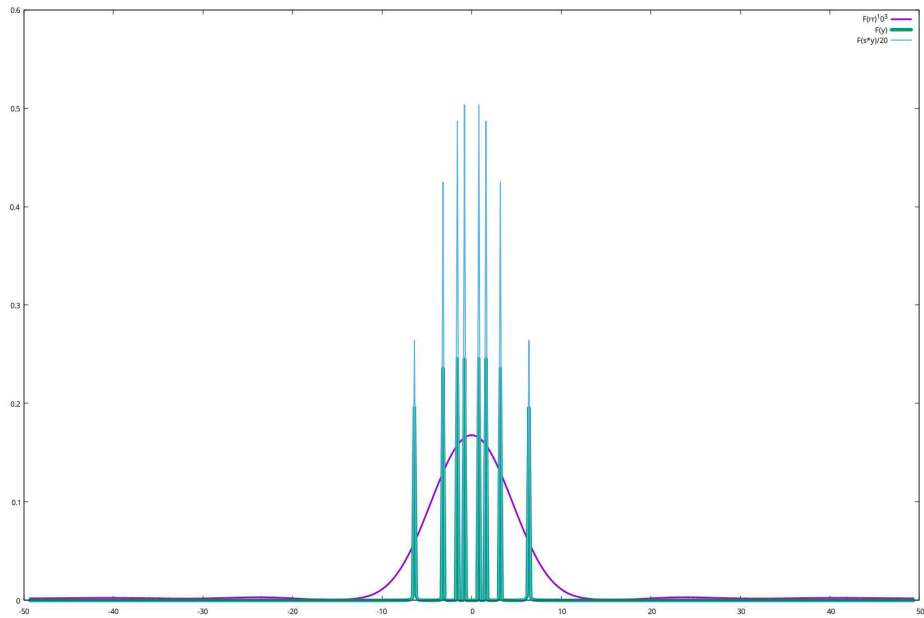
Podczas pomiarów powstaje splot funkcji wejściowej y i funkcji reakcji aparatury r . Jeżeli r jest zadana dla $-m_r \leq k \leq m_r$, wówczas

$$(y * r)_l = \sum_{k=-m_r}^{m_r} y_{l-k} r_k. \quad (7.6)$$

Najwygodniej jest zdefiniować funkcję r nie dla czasu dodatniego i ujemnego, ale dla przedziału czasu T dla którego jest zdefiniowana funkcja y . Wówczas wartości funkcji r odpowiadające ujemnemu czasowi należy przesunąć o T . W najprostrzym przypadku $r_0 = 1$, $r_{i \neq 0} = 0$ splot jest funkcją wejściową.

7.1.1 Przykład

Jeżeli $r_{(-m_a \leq i \leq m_a)} = \cos\left(\frac{2.1i}{2\pi}\right) + 0.5$, $r_{(i < -m_a, i > m_a)} = 0$, $m_a = 3$, a funkcja wejściowa jest dana równaniem 7.3, wówczas transformaty funkcji g i splotu $s * r$ są takie jak na rysunku 7.4.



Rysunek 7.4: Unormowana połowa transformaty funkcji reakcji r , transformaty funkcji y , oraz i transformaty splotu funkcji, $y * r$.

Rozdział 8

Minimalizacja (algorytm Levenberga - Marquardta)

Niech dopasowywana funkcja ma postać $y = f(t, \mathbf{c} = c_1, c_2, \dots, c_M)$, gdzie t jest zmienną niezależną, natomiast c_j są dopasowywanymi parametrami. Jeżeli liczba punktów pomiarowych wynosi N , wówczas

$$\chi^2(\mathbf{c}) = \sum_{i=1}^{i=N} \left[\frac{y_i - f(t_i, \mathbf{c})}{\sigma_i} \right]^2, \quad (8.1)$$

gdzie σ_i jest dokładnością z jaką została zmierzona wartość y_i . Jeżeli błędy pomiarowe nie są znane, należy dla wszystkich punktów przyjąć $\sigma = 1$. Pozwala to znaleźć dopasowanie, ale bez możliwości wykonania testu chikwadrat.

Procedura dopasowania wymaga obliczenia pochodnych χ^2 po parametrach c_l . Po dwukrotnym zróżniczkowaniu równania 8.1 widać, że powinny zostać obliczone: pierwsza pochodna $\frac{\delta f}{c_j}$, oraz druga pochodna $\frac{\delta^2 f}{\delta c_j \delta c_k}$. Jednak druga pochodna jest mnożona przez małą różnicę $y_i - f(t_i, \mathbf{c})$ i w praktyce można ją pominąć. Można więc zapisać

$$\frac{\partial^2 \chi^2}{\partial c_j \partial c_k} = \sum_{i=1}^{i=N} \frac{1}{\sigma_i^2} \left[\frac{\partial f(t_i, \mathbf{c})}{\partial c_j} \frac{\partial f(t_i, \mathbf{c})}{\partial c_k} \right] = 2A_{jk}. \quad (8.2)$$

Procedura znajdowania $\mathbf{c}_{\min}$ polega na rozwiązywaniu układu równań

$$\sum_{k=1}^{k=M} A_{jk} \delta c_k = -0.5 \frac{\delta \chi^2}{\delta c_j} = B_j \quad (8.3)$$

dla poprawek δc_k do aktualnych wartości parametrów c_k . Poprawione wartości parametrów należy potraktować jako wartości wyjściowe i obliczyć kolejne poprawki.

Opisane podejście najlepiej połączyć z metodą najbardziej stromego schodzenia $\delta c_k = C B_k$, gdzie C jest stałą, którą trzeba dobrać. W celu połączenia metod macierz $\mathbf{A}$ należy zastąpić przez jej zmodyfikowaną wersję $\mathbf{A}'$ spełniającą warunek

$$\begin{aligned} A'_{jk} &\equiv A_{jk}(1 + \zeta) \quad (\text{dla } j = k) \\ A'_{jk} &\equiv A_{jk} \quad (\text{dla } j \neq k) \end{aligned} \quad (8.4)$$

i rozwiązywać równanie

$$\sum_{k=1}^{k=M} A'_{jk} \delta c_k = B_j. \quad (8.5)$$

Kluczowy jest dobór wartości ζ . Na początku dobrze przyjąć jakąś małą wartość, może być 0.001. Dalsze kroki powinny być następujące:

1. Rozwiązać równanie 8.5 i dodać do parametrów a_k znalezione poprawki.
2. Obliczyć funkcję $\chi^2(\mathbf{c} + \delta \mathbf{c})$.
3. Pomnożyć ζ przez 10 jeżeli $\chi^2(\mathbf{c} + \delta \mathbf{c}) \geq \chi^2(\mathbf{c})$, albo podzielić ζ przez 10 jeżeli $\chi^2(\mathbf{c} + \delta \mathbf{c}) < \chi^2(\mathbf{c})$.
4. Jeżeli zmiana χ^2 jest ujemna i odpowiednio mała (według uznania) obliczenia są zakończone. W przeciwnym wypadku trzeba przejść do punktu 1 i obliczyć kolejne przybliżenie.

Macierz $\mathbf{A}^{-1}$ jest macierzą kowariancji, więc pierwiastki kwadratowe z wartości na przekątnej wskazują dokładność dopasowania parametrów. Można także wykonać test chikwadrat w celu określenia jakości dopasowania. W szczególnym przypadku kiedy nie są znane błędy pomiarowe i zostało przyjęte $\sigma_i = 1$ dokładność dopasowanych wartości współczynników określa się inaczej.

Procedura jest bardzo ogólna i pozwala dopasowywać nie tylko krzywe, ale i obiekty o większej liczbie wymiarów. W ogólnym przypadku każdy t_i jest wektorem. Jeżeli dopasowujemy powierzchnię w 3D, wówczas $t_i = (x_i, y_i)$.

Wskazówki

Wartości pochodnych najlepiej obliczać numerycznie. Przy równomiernie rozmieszczonych punktach dwie pierwsze pochodne dowolnej funkcji $f(x)$ można

przybliżyć jako

$$\frac{df}{dx} \simeq \frac{f(x+dx) - f(x-dx)}{2dx}, \quad (8.6)$$

oraz

$$\frac{d^2f}{dx^2} \simeq \frac{f(x+dx) - 2f(x) + f(x-dx)}{dx^2} \quad (8.7)$$

Przy pisaniu programu warto wydzielić z głównego bloku procedury do:

- obliczania wartości dopasowywanej funkcji,
- obliczania wartości funkcji χ^2 ,
- obliczania pierwszych pochodnych χ^2 ,
- odwracania macierzy, oraz
- rozwiązywania układu równań np. metodą eliminacji Gaussa-Jordana.

Główny blok powinien pytać o nazwę zbioru z danymi i wczytać je, wyświetlić informację jaką funkcja będzie dopasowywana, zapytać o wartości początkowe dopasowywanych parametrów i na końcu wypisać dopasowane wartości.

Przekazywanie funkcji jako argumentu funkcji można zrealizować jak w poniższym programie.

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <functional>
using namespace std;

double fun(double x){
    return sin(x);
}

void wywolaj(function<double(double)> f, double arg){
    double wynik = f(arg);
    cout << wynik << endl;
}

int main(int argc, char **argv){
    double pi=M_PI;
    wywolaj(fun, pi/2.0);
    return 0; }
```

Ćwiczenie. Proszę napisać program dopasowujący funkcję jednej zmiennej do zestawu punktów. Postać funkcji można wybrać według uznania.

Rozdział 9

Równania różniczkowe zwyczajne

Metoda rozwiązywania równania, albo ich układu zależy od typu problemu. Może to być zagadnienie warunku początkowego, albo warunków brzegowych.

9.1 Zagadnienie warunku początkowego

Równanie różniczkowe zwyczajne pierwszego stopnia jednej zmiennej ma postać

$$\frac{dy}{dt} = f(t, y). \quad (9.1)$$

Równanie wyższego stopnia sprowadza się do układu równań pierwszego stopnia. Ogólnie układ n równań różniczkowych zwyczajnych można zapisać jako

$$\frac{dy_i}{dt} = f_i(t, y_0, \dots, y_{n-1}), \quad (9.2)$$

gdzie t jest zmienną niezależną.

W typowym zagadnieniu warunku początkowego, jakim jest spadek w polu grawitacyjnym równanie ruchu ma postać

$$m \frac{d^2 z}{dt^2} = -mg + Cv^2, \quad (9.3)$$

gdzie z jest wysokością, a v prędkością. Równanie to można zapisać jako układ

$$\begin{aligned} \frac{dz}{dt} &= v \equiv f_1(t, z, v) \\ \frac{dv}{dt} &= -g + \frac{C}{m}v^2 \equiv f_2(t, z, v). \end{aligned} \quad (9.4)$$

Rozwiązanie problemu polega na znalezieniu zależności $z(t)$ i $v(t)$. Jeżeli wartości zmiennej z w chwilach t i $t + dt$ zostaną oznaczone odpowiednio z_n i z_{n+1} , wówczas można napisać

$$\begin{aligned} z_{n+1} &= z_n + dt f_1(t_n, z_n, v_n) \\ v_{n+1} &= v_n + dt f_2(t_n, z_n, v_n). \end{aligned} \quad (9.5)$$

Na początku należy przyjąć $z_n = z(t_0)$ i $v_n = v(t_0)$ i obliczyć z_{n+1} i v_{n+1} , a w kolejnych krokach podstawiać $z_{n+1} \rightarrow z_n$ i $v_{n+1} \rightarrow v_n$. Metoda ta niestety jest niestabilna.

9.1.1 Metoda Runge-Kutty 4 rzędu

Dla równania

$$\frac{dy}{dt} = f(t, y) \quad (9.6)$$

należy zapisać

$$\begin{aligned} k_1 &= dt f(t_n, y_n) \\ k_2 &= dt f\left(t_n + \frac{dt}{2}, y_n + \frac{k_1}{2}\right) \\ k_3 &= dt f\left(t_n + \frac{dt}{2}, y_n + \frac{k_2}{2}\right) \\ k_4 &= dt f(t_n + dt, y_n + k_3) \\ y_{n+1} &= y_n + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} \end{aligned} \quad (9.7)$$

Ćwiczenie. Proszę rozwiązać równanie spadku przedmiotu o masie $m = 10 \text{ kg}$ z wysokości $z_0 = 100 \text{ m}$, jeżeli współczynnik oporu $C = 1$, a przyspieszenie grawitacyjne $g = 10 \text{ m s}^{-2}$.

9.1.2 Metoda Fehlberga^G

Istnieje udoskonalona wersja metody opisanej w rozdziale 9.1.1. Przy użyciu 2 zestawów współczynników można uzyskać teoretyczną dokładność albo taką samą jak w metodzie Runge-Kutty 4 rzędu, albo o rząd lepszą. Zamiast

równań 9.1.1 należy jest

$$\begin{aligned}
k_1 &= dt f(t_n, y_n) \\
k_2 &= dt f(t_n + a_2 dt, y_n + b_{2,1} k_1) \\
k_3 &= dt f(t_n + a_3 dt, y_n + b_{3,1} k_1 + b_{3,2} k_2) \\
k_4 &= dt f(t_n + a_4 dt, y_n + b_{4,1} k_1 + b_{4,2} k_2 + b_{4,3} k_3) \\
k_5 &= dt f(t_n + a_5 dt, y_n + b_{5,1} k_1 + b_{5,2} k_2 + b_{5,3} k_3 + b_{5,4} k_4) \\
k_6 &= dt f(t_n + a_6 dt, y_n + b_{6,1} k_1 + b_{6,2} k_2 + b_{6,3} k_3 + b_{6,4} k_4 + b_{6,5} k_5) \\
y_{n+1} &= y_n + c_1 k_1 + c_2 k_2 + c_3 k_3 + c_4 k_4 + c_5 k_5 + c_6 k_6 \\
y_{n+1}^* &= y_n + c_1^* k_1 + c_2^* k_2 + c_3^* k_3 + c_4^* k_4 + c_5^* k_5 + c_6^* k_6
\end{aligned} \tag{9.8}$$

Jest to metoda Fehlberga. Oryginalne współczynniki zostały później ulepszone. Zmodyfikowana metoda jest znana jako metoda Cash-Karp.

Tabela 9.1: Współczynniki Cash-Karp. W wierszach 1 - 6 w pierwszej kolumnie są współczynniki a_j , a w kolejnych w kolumnach współczynniki $b_{i,j}$, gdzie i oznacza numer kolumny, natomiast j numer wiersza. Współczynnik a_1 jest zerowy, ponieważ nie występuje w równaniu 9.8. W wierszu 7 są współczynniki c_k , natomiast w wierszu 8 współczynniki c_k^* . Indeks k ma wartości 1 - 6.

0						
1/5	1/5					
3/10	3/40	9/40				
3/5	3/10	-9/10	6/5			
1	-11/54	5/2	-70/27	35/27		
<u>7/8</u>	<u>1631/55296</u>	<u>175/512</u>	<u>575/13824</u>	<u>44275/110592</u>	<u>253/4096</u>	
	<u>37/378</u>	0	<u>250/621</u>	<u>125/594</u>	0	<u>512/1771</u>
	<u>2825/27648</u>	0	<u>18575/48384</u>	<u>13525/55296</u>	<u>277/14336</u>	<u>1/4</u>

Wartość y_{n+1}^* ma teoretyczną dokładność 4 rzędu, a y_{n+1} 5 rzędu.

Jeżeli wynik dokładniejszy zostanie uznany za dokładny, różnica $\Delta = y_{n+1} - y_{n+1}^*$ stanie się miarą błędu. Można jej użyć do automatycznej regulacji kroku całkowania. Jeżeli Δ_0 jest założoną akceptowaną dokładnością, a dt_0 jest długością aktualnego kroku, to następny krok należy wykonać przyjmując

$$dt = dt_0 \left| \frac{\Delta_0}{\Delta} \right|^{0.2}. \tag{9.9}$$

Istotnym problemem jest niemożność skorygowania błędu popełnionego na w wykonanym kroku całkowania. Algorytm nie przewiduje żadnego cofania. Obliczone y_{n+1} odpowiadające $t+dt$ już nie jest zmieniane. Korekta długości kroku dotyczy następnego etapu od $t+dt$ do $t+2dt$. Jeżeli przy całkowaniu od t do $t+dt_0$ oszacowany błąd wynosi $\Delta \gg \Delta_0$ można tylko przerwać obliczenia i uruchomić program jeszcze raz. Dlatego dobrze jest zaczynać całkowanie z małym krokiem i pozwolić programowi na jego wydłużanie.

Ćwiczenie. Proszę rozwiązać równanie spadku przedmiotu o masie $m = 10 \text{ kg}$ z wysokości $z_0 = 100 \text{ m}$, jeżeli współczynnik oporu $C = 1$, a przyspieszenie grawitacyjne $g = 10 \text{ m s}^{-2}$.

Rozdział 10

Równania różniczkowe cząstkowe - metody różnic skończonych

W przypadku równań różniczkowych cząstkowych wybór metody zależy od rodzaju równania: hiperboliczne (równanie falowe), paraboliczne (równanie dyfuzji), czy eliptyczne (równanie Poissona). Jeżeli równanie ma postać

$$f_5(x_1, x_2) \frac{\partial^2 w}{\partial x_1^2} + f_6(x_1, x_2) \frac{\partial^2 w}{\partial x_2^2} + f_7(x_1, x_2) \frac{\partial^2 w}{\partial x_1 \partial x_2} + \left(f_1(x_1, x_2) + f_2(x_1, x_2)w + f_3(x_1, x_2) \frac{\partial w}{\partial x_1} + f_4(x_1, x_2) \frac{\partial w}{\partial x_2} \right) = 0 \quad (10.1)$$

jest ono hiperboliczne gdy $4f_7 - f_6f_5 > 0$, paraboliczne gdy $4f_7 - f_6f_5 = 0$ i eliptyczne gdy $4f_7 - f_6f_5 < 0$ (np. Wikipedia). Stosowane algorytmy można podzielić na *implicit*, *semi implicit*, oraz *explicit*. Tych ostatnich należy unikać. Jeżeli ośrodek ma prosty kształt, który nie ulega zmianom można rozwiązać równanie dla sztywnej siatki równoodległych punktów (metoda różnic skończonych).

10.1 Równania paraboliczne

W tym rozdziale rozwiązywanie równań parabolicznych jest omówione na przykładzie równania dyfuzji. Może ono dotyczyć dyfuzji gazu, jak również przenikania ciepła.

W sytuacji kiedy własności materiału są stałe, szybkość dyfuzji ciepła opisuje równanie

$$\frac{\partial T}{\partial t} = \kappa \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} \right), \quad \kappa = \frac{\lambda}{\rho c_w} \quad (10.2)$$

gdzie λ oznacza przewodnictwo cieplne, ϱ gęstość, a c_w ciepło właściwe.

W bardziej ogólnym przypadku kiedy $\kappa = \kappa(t, x, y, z)$ równanie 10.2 ma postać

$$\frac{\partial T}{\partial t} = \nabla \cdot (\kappa \nabla T), \quad (10.3)$$

albo gdyby dyfuzja była anizotropowa

$$\frac{\partial T}{\partial t} = \sum_{k=1}^3 \sum_{l=1}^3 \frac{\partial}{\partial x_k} \left(\kappa_{kl} \frac{\partial T}{\partial x_l} \right). \quad (10.4)$$

W dalszej części rozdziału zostały przedstawione metody rozwiązywania równania 10.2 przy różnych warunkach brzegowych, w jednym wymiarze i w dwu wymiarach. Opisany algorytm jest typu *implicit*. Można użyć także metody Crank-Nicolson typu *semi implicit*. Oba algorytmy są stabilne dla dowolnych kroków całkowania dt .

10.1.1 Problem jednowymiarowy

Jeżeli transport odbywa się tylko wzdłuż osi z , na przykład między równoległymi płaszczyznami A i B równanie 10.2 można zapisać w postaci

$$\frac{T_j^{n+1} - T_j^n}{dt} = \frac{\kappa}{dx^2} (T_{j+1}^{n+1} - 2T_j^{n+1} + T_{j-1}^{n+1}), \quad (10.5)$$

gdzie górne indeksy określają czas, a dolne współrzędną przestrzenną. Indeks n tak jak wcześniej oznacza wartości w chwili t , a indeks $n+1$ chwilę $t+dt$. Indeksy $j-1$, j i $j+1$ oznaczają wartości w sąsiednich punktach siatki numerycznej oddległych od siebie o dz . Równanie 10.5 można przekształcić do postaci

$$T_{j+1}^{n+1} \left(-\frac{\kappa dt}{dz^2} \right) + T_j^{n+1} \left(1 + \frac{2\kappa dt}{dz^2} \right) + T_{j-1}^{n+1} \left(-\frac{\kappa dt}{dz^2} \right) = T_j^n \quad (10.6)$$

w której wszystkie niewiadome, czyli wartości temperatury w chwili $t+dt$ są po lewej stronie i muszą być obliczone równocześnie. Jest to podejście typu *implicit*. Łatwo zauważyć, że równań 10.5 i 10.6 nie można zastosować dla punktów leżących na płaszczyznach A i B, czyli ogólnie na brzegach.

Warunki brzegowe można mogą być sformułowane na różne sposoby. Przy dyfuzji ciepła może być znana temperatura na powierzchniach granicznych, albo strumień energii przez te powierzchnie. Kiedy rozważany jest transport ciepła między równoległymi płaszczyznami, trzeba sformułować równania dla:

1. Temperatur na powierzchniach granicznych, w przypadku jednowymiarowym (często oznaczanym 1D) T_0 na płaszczyźnie A i T_m na płaszczyźnie B, albo
2. Gradientów temperatury przy płaszczyznach granicznych, w przypadku 1D odpowiednio $(T_1 - T_0)/dz$ i $(T_m - T_{m-1})/dz$.

W obu przypadkach algorytm może pozostać *implicit*.

Warunek brzegowy na temperaturę

Jeżeli znana jest temperatura na powierzchniach ograniczających ośrodek, wówczas warunki brzegowe można zapisać w postaci 2 równań:

$$T_2^{n+1} \left(-\frac{\kappa dt}{dz^2} \right) + T_1^{n+1} \left(1 + \frac{2\kappa dt}{dz^2} \right) = T_1^n - T_0^{n+1} \left(-\frac{\kappa dt}{dz^2} \right), \quad (10.7)$$

$$T_{m-1}^{n+1} \left(1 + \frac{2\kappa dt}{dz^2} \right) + T_{m-2}^{n+1} \left(-\frac{\kappa dt}{dz^2} \right) = T_{m-1}^n - T_m^{n+1} \left(-\frac{\kappa dt}{dz^2} \right), \quad (10.8)$$

Ostatecznie, dla $m+1$ punktów od $j=0$ do $j=m$ trzeba rozwiązać układ $m-1$ równań: 10.7, 10.8, oraz (dla $2 \leq j \leq m-2$) równanie 10.6. Układ ten można zapisać jako równanie wektorowe

$$\begin{bmatrix} b & a & 0 & \cdots & \cdots & \cdots & \cdots & \cdots & 0 \\ c & b & a & \cdots & \cdots & \cdots & \cdots & \cdots & 0 \\ & & & & \vdots & & & & \\ 0 & \cdots & 0 & c & b & a & 0 & \cdots & 0 \\ & & & & \vdots & & & & \\ 0 & \cdots & \cdots & \cdots & \cdots & 0 & c & b & a \\ 0 & \cdots & \cdots & \cdots & \cdots & 0 & 0 & c & b \end{bmatrix} \begin{bmatrix} T_{m-1}^{n+1} \\ T_{m-2}^{n+1} \\ \vdots \\ T_j^{n+1} \\ \vdots \\ T_2^{n+1} \\ T_1^{n+1} \end{bmatrix} = \begin{bmatrix} T_{m-1}^n - (T_m^{n+1})c \\ T_{m-2}^n \\ \vdots \\ T_j^n \\ \vdots \\ T_2^n \\ T_1^n - (T_0^{n+1})a \end{bmatrix} \quad (10.9)$$

gdzie

$$\begin{aligned} a &= c = \left(-\frac{\kappa dt}{dz^2} \right) \\ b &= \left(1 + \frac{2\kappa dt}{dz^2} \right). \end{aligned} \quad (10.10)$$

Ćwiczenie. Proszę zbadać ewolucję temperatury w jednorodnej płaskiej warstwie kiedy temperatura na jednej powierzchni jest stała $T_0 = T_A$, natomiast na drugiej jest sinusoidalne zmienna $T_m = T_B + C \sin(2\pi t/\tau)$. Niech grubość warstwy $L = 10$ m, oraz okres $\tau = 1$ rok, $T_A = 10^\circ$ C, $T_B = 10^\circ$, $C = 20^\circ$ C.

Wartości λ , c i ϱ proszę przyjąć takie jak dla piasku. Na jakiej głębokości temperatura nigdy nie spada poniżej zera?

Wskazówka. W przypadku półprzestrzeni można znaleźć rozwiązanie analityczne. Dla $\varrho = 1600 \text{ kg m}^{-3}$, $\lambda = 1 \text{ W m}^{-1} \text{ K}^{-1}$ i $c_w = 830 \text{ J kg}^{-1}$ analitycznie obliczona głębokość zamarzania gruntu wynosi $\simeq 1.9059 \text{ m}$.

Warunek brzegowy na strumień

Zamiast temperatur powierzchni ograniczających może być znany strumień przez te powierzchnie. W takim przypadku trzeba dodać fikcyjne punkty znajdujące się poza ośrodkiem, po jednym przy każdej powierzchni. Temperatura w tych punktach jest funkcją wartości strumienia.

Dla przykładu niech strumień ciepła przez płaszczyznę A jest zerowy, a strumień przez powierzchnię B wynosi $F(t)$. Zerowy strumień przez powierzchnię A oznacza, że temperatura w fikcyjnym punkcie przy tej powierzchni $T_{-1} = T_1$. Równanie na temperaturę na tej powierzchni ma w takim przypadku postać

$$(c + a)T_1^{n+1} + bT_0^{n+1} = T_0^n \quad (10.11)$$

Gdy strumień przez powierzchnię B wynosi F , warunek na temperaturę na tej powierzchni ma postać

$$F^{n+1} = \frac{\lambda}{2dz} (T_{m+1}^{n+1} - T_{m-1}^{n+1}), \quad (10.12)$$

więc

$$bT_m^{n+1} + (c + a)T_{m-1}^{n+1} = T_m^n - c \frac{2dz}{\lambda} F^{n+1}. \quad (10.13)$$

Ostatecznie równanie 10.9 przybiera postać

$$\begin{bmatrix} b & (a+c) & 0 & \dots & \dots & \dots & \dots & 0 \\ c & b & a & \dots & \dots & \dots & \dots & 0 \\ & & & \vdots & & & & \\ 0 & \dots & 0 & c & b & a & \dots & 0 \\ & & & \vdots & & & & \\ 0 & \dots & \dots & \dots & \dots & c & b & a \\ 0 & \dots & \dots & \dots & \dots & 0 & (a+c) & b \end{bmatrix} \begin{bmatrix} T_m^{n+1} \\ T_{m-1}^{n+1} \\ \vdots \\ T_j^{n+1} \\ \vdots \\ T_1^{n+1} \\ T_0^{n+1} \end{bmatrix} = \begin{bmatrix} T_m^n - c \frac{2dz}{\lambda} F^{n+1} \\ T_{m-1}^n \\ \vdots \\ T_j^n \\ \vdots \\ T_1^n \\ T_0^n \end{bmatrix} \quad (10.14)$$

10.2 Zadania treningowe

1. Napisać program obliczający wartość funkcji $\sqrt{\cdot}(x)$ korzystając ze wzoru na rozwinięcie Taylora tej funkcji wokół jedynki. Narysować wykres rzędów rozwinięcia 2, 3, 5, i 10 i porównać z wykresem prawdziwej funkcji.

2. Rozważmy ruch planet wokół Słońca. Zgodnie z prawem Keplera odbywa się on po elipsach. Parametrem opisującym spłaszczenie elipsy jest mimośród

$$\epsilon = (1 - b^2/a^2)^{0.5}, \quad (10.15)$$

gdzie a, b to półosie elipsy. Dla Ziemi $\epsilon = 0.0167$, a dla Merkurego (0.2056). Położenie planety w czasie t (liczonym od peryhelium) jest

$$x = a(\cos(E) - \epsilon) \quad (10.16)$$

$$y = a(1 - \epsilon \sin(E)) \quad (10.17)$$

gdzie anomalia mimośrodowa E spełnia równanie Keplera

$$E - \epsilon \sin(E) = 2\pi \frac{t}{T} \quad (10.18)$$

gdzie T to okres obiegu. Z równania Keplera można wyznaczyć wartość anomalii mimośrodowej dla danego czasu mierzonego od chwili największego zbliżenia do Słońca (jest to równanie nieliniowe, które możemy rozwiązać jedną z poznanych metod), a następnie znajdujemy położenie z pierwszego równania. Proszę narysować wykres położenia Ziemi w ciągu bieżącego roku, jeżeli punkt największego zbliżenia wypadł 5 stycznia 2020 oznacz położenie na dzień dzisiejszy. Wykres może być wyskalowany w jednostkach astronomicznych lub mln km.

3. Jednym z rozkładów prawdopodobieństwa, jest rozkład normalny (Gaussa). W tak zwanej postaci standardowej ($\sigma = 1, \mu = 0$), gęstość prawdopodobieństwa tego rozkładu ma postać

$$\varrho(x) = \frac{1}{\sqrt{2\pi}} \exp(-x^2/2). \quad (10.19)$$

Dystrybuanta rozkładu to funkcja opisująca prawdopodobieństwo P , że zmienna losowa X przyjmie wartość $\leq x$:

$$\Phi(x) = P(X \leq x) = \int_x^\infty -\varrho(z) dz. \quad (10.20)$$

Calka $\int \exp(-x^2/2)dx$, jest niemożliwa do wyznaczenia analitycznego. Należy napisać program, który będzie znajdował numerycznie, z użyciem jednej z metod całkowania, wartość dystrybuanty dla zadanej wartości x . Dobrać n tak, aby zapewnić odpowiednią dokładność wyników.

```
import scipy.stats as stats
```

```
PythonIntegral = stats.norm.cdf (x=+1, loc=0, scale=1) - stats.norm.cdf  
(x=-1, loc=0, scale=1) Uwaga. Liczy calke z funkcji  $\exp(-x^2/2)/\sqrt{2 * \pi}$ 
```

4. Proszę rozwiązać problem ruchu oscylatora harmonicznego za pomocą:

- a) schematu Eulera,
- b) zmodyfikowanego schematu Eulera,
- c) schematu Rungego-Kutty 4 stopnia.

Dla każdej metody należy porównać położenie i prędkość po 2 okresach z rozwiązaniem dokładnym (analitycznym), sprawdzić zależność dokładności od kroku h .

5. Równanie Lotki-Volterry opisuje zmianę w czasie liczby drapieżników l (np. lisów) i ich ofiar k (np. królików) w danym ekosystemie. Odpowiedni układ równań jest następujący

$$\begin{aligned}\frac{dk}{dt} &= (\alpha - \beta l)k \\ \frac{dl}{dt} &= (\delta k - \gamma)l,\end{aligned}\tag{10.21}$$

gdzie α to naturalny przyrost populacji ofiar β to częstość ubywania ofiar w wyniku zjadania przez drapieżniki, δ to częstość narodzin drapieżników, natomiast γ to częstość ubywania drapieżników. Należy rozwiązać równanie 10.21 za pomocą wybranej metody ($\alpha = 2.0$, $\beta = 0.2$, $\delta = 0.01$, $\gamma = 0.2$).

6. Problem wahadła matematycznego

$$\frac{d^2\theta}{dt^2} + gl \sin(\theta) = 0,\tag{10.22}$$

bez przybliżenia małych wychyleń. Należy narysować wykres fazowy (położenie w funkcji pędu) dla różnych warunków początkowych.

Bibliografia

- Press, W.H. et al., 2003. Numerical Recipes in Fortran 77 Second Edition: The Art of Scientific Computing, Cambridge University Press.
- Baldanzi, L., Crocetti, L., Falaschi, F., Bertolucci, M., Belli, J., Fanucci, L. and Sergio Saponara, 2020. Cryptographically Secure Pseudo-Random Number Generator IP-Core Based on SHA2 Algorithm. Sensors 2020, 20, 1869; doi:10.3390/s20071869
- Gortas et al., 2011. Icarus 212, 858-866
- L'Ecuyer and Simard, R., 2007. TestU01: A C Library for Empirical Testing of Random Number Generators. ACM Transactions on Mathematical Software, Vol. 33, No. 4, Article 22. Doi: 10.1145/1268776.1268777
- Park, S.K., Miller, K.W., 1988. Random number generators: good ones are hard to find. Communications of the ACM 31, no. 10, 1192-1201